

joyerIA — System Architecture & Tech Stack Document

1. INFORMACIÓN GENERAL

Proyecto

joyerIA

Documento

System Architecture & Tech Stack Document

Objetivo

Definir la arquitectura técnica oficial del MVP de joyerIA.

Este documento establece:

- Arquitectura general.
- Estructura frontend.
- Arquitectura backend.
- Stack tecnológico.
- Estrategia de escalabilidad.
- Seguridad.
- Realtime.
- Integración IA.
- Flujo de datos.
- Infraestructura.
- Buenas prácticas.

2. PRINCIPIOS ARQUITECTÓNICOS

Objetivos principales

La arquitectura debe priorizar:

- Escalabilidad progresiva.
- Mantenibilidad.
- Simplicidad inteligente.

- UX premium.
 - Bajo costo inicial.
 - Desarrollo rápido.
 - Modularidad.
 - Seguridad.
 - Performance.
-

Filosofía técnica

El MVP NO debe construirse utilizando:

- Microservicios complejos.
- Kubernetes.
- Arquitectura distribuida innecesaria.
- Event-driven enterprise.

La prioridad inicial será:

- Monolito modular.
 - Arquitectura limpia.
 - Feature-first.
 - Servicios desacoplados.
-

3. ARQUITECTURA GENERAL DEL SISTEMA

```
Mobile App (Flutter)
  ↓
API / Supabase Layer
  ↓
Application Services
  ↓
PostgreSQL + Storage + Realtime
  ↓
External Services (AI / Notifications)
```

4. STACK TECNOLÓGICO OFICIAL

Frontend Mobile

Tecnología

Flutter.

Lenguaje

Dart.

Razones

- Excelente rendimiento.
 - UI premium.
 - Animaciones fluidas.
 - Mobile-first.
 - Cross-platform.
 - Excelente experiencia visual.
-

State Management

Tecnología

Riverpod.

Razones

- Escalable.
 - Testable.
 - Bajo acoplamiento.
 - Compatible con Clean Architecture.
-

Routing

Tecnología

GoRouter.

Razones

- Deep links.
 - Guards.
 - Navegación escalable.
-

Backend

Tecnología principal

Supabase.

Servicios utilizados

- PostgreSQL.
 - Auth.
 - Realtime.
 - Storage.
 - Edge Functions.
-

Base de datos

Tecnología

PostgreSQL.

Razones

- Excelente soporte relacional.
 - Escalable.
 - SQL avanzado.
 - Integridad de datos.
 - Ideal para sistemas sociales.
-

Realtime

Tecnología

Supabase Realtime.

Uso

- Chat.
 - Notificaciones en vivo.
 - Actualización de mensajes.
-

Push Notifications

Tecnología

Firebase Cloud Messaging (FCM).

IA

Tecnología inicial

OpenAI APIs.

Funciones

- Generación de imágenes.
 - Variaciones.
 - Asistencia creativa.
-

Storage

Tecnología

Supabase Storage.

Contenido

- Imágenes.
 - Videos.
 - Archivos.
 - Documentos.
-

Analytics

Tecnología sugerida

PostHog.

Uso

- Product analytics.
 - Eventos.
 - Funnels.
 - Retención.
-

Crash Reporting

Tecnología

Firebase Crashlytics.

CI/CD

Tecnología

GitHub Actions.

5. ARQUITECTURA FRONTEND

Filosofía frontend

La app debe utilizar:

- Clean Architecture.
 - Feature-first architecture.
 - Modularidad.
 - Reutilización.
 - UI desacoplada.
-

Estructura principal

```
lib/  
├─ core/  
├─ shared/  
├─ features/  
├─ app/  
└─ main.dart
```

core/

Responsabilidades globales.

```
core/  
├─ config/  
├─ constants/  
├─ network/  
├─ router/  
├─ services/  
├─ theme/  
├─ security/  
├─ storage/  
├─ errors/  
└─ utils/
```

shared/

Componentes reutilizables.

```
shared/  
├─ widgets/  
├─ components/  
├─ models/  
├─ ui/  
└─ helpers/
```

features/

Módulos de negocio.

```
features/  
├─ auth/  
├─ feed/  
├─ profile/  
├─ chat/  
├─ ai_designer/  
├─ upload/  
├─ notifications/  
├─ search/  
├─ verification/  
└─ admin/
```

Arquitectura interna de cada feature

```
feature/  
├─ data/  
├─ domain/  
├─ presentation/  
└─ module.dart
```

data/

Acceso a datos.

```
data/  
├─ datasource/  
├─ repositories/  
├─ dto/  
└─ mappers/
```

domain/

Lógica de negocio.

```
domain/  
├─ entities/  
├─ repositories/  
├─ usecases/  
└─ services/
```

presentation/

UI + estado.

```
presentation/  
├─ pages/  
├─ widgets/  
├─ providers/  
├─ controllers/  
└─ states/
```

Flujo frontend recomendado

```
UI  
↓  
Provider  
↓  
UseCase  
↓  
Repository  
↓  
Datasource  
↓  
Supabase/API
```

6. ARQUITECTURA BACKEND

Estrategia backend

El backend seguirá arquitectura:

- Modular.
 - Desacoplada.
 - Escalable.
 - Monolito modular.
-

Arquitectura lógica backend

```
backend/  
├── modules/  
│   ├── auth/  
│   ├── users/  
│   ├── feed/  
│   ├── chat/  
│   ├── ai/  
│   ├── moderation/  
│   ├── notifications/  
│   └── verification/
```

Arquitectura por capas

```
API Layer  
↓  
Application Layer  
↓  
Domain Layer  
↓  
Infrastructure Layer  
↓  
Database / Storage / External APIs
```

API Layer

Responsable de:

- Endpoints.
 - Auth validation.
 - Request parsing.
 - Response formatting.
-

Application Layer

Responsable de:

- Casos de uso.
 - Coordinación.
 - Lógica de aplicación.
-

Domain Layer

Responsable de:

- Reglas de negocio.
 - Entidades.
 - Lógica central.
-

Infrastructure Layer

Responsable de:

- PostgreSQL.
 - Storage.
 - APIs externas.
 - Realtime.
 - IA.
-

7. BASE DE DATOS

Tecnología

PostgreSQL.

Estrategia

La base de datos será:

- Relacional.
 - Normalizada.
 - Optimizada mediante índices.
 - Escalable.
-

Entidades principales

```
users
profiles
posts
comments
likes
saved_posts
messages
conversations
notifications
verification_requests
ai_generations
reports
```

Estrategias importantes

Soft delete

Eliminar lógicamente entidades importantes.

UUIDs

Utilizar UUIDs como IDs principales.

Indexing

Indexar:

- feed.
 - búsquedas.
 - chats.
 - relaciones frecuentes.
-

8. STORAGE Y MULTIMEDIA

Estrategia multimedia

El sistema debe optimizar:

- imágenes.
 - videos.
 - uploads.
-

Flujo multimedia

```
Upload
↓
Compression
↓
Storage
↓
CDN Delivery
```

Reglas multimedia

Videos

- Duración limitada.
 - Compresión automática.
 - Streaming optimizado.
-

Imágenes

- Resize automático.
 - WebP cuando sea posible.
-

9. REALTIME ARCHITECTURE

Realtime utilizado en

- Chat.
 - Mensajes.
 - Estado online.
 - Notificaciones.
-

Arquitectura realtime

```
Client
↔ Realtime Channel
↔ Supabase Realtime
↔ PostgreSQL Changes
```

10. IA ARCHITECTURE

Principio clave

La IA debe mantenerse desacoplada del core principal.

Arquitectura IA

```
Mobile App
↓
AI Service Layer
↓
OpenAI APIs
↓
Result Processing
↓
Storage
```

Responsabilidades IA

AI Service Layer

Responsable de:

- prompts.
- rate limiting.
- control de costos.
- validaciones.
- retries.

Estrategia futura

La IA podrá evolucionar posteriormente hacia:

- Workers dedicados.
- Queue system.
- Rendering pipeline.
- Modelos propios.

11. SEGURIDAD

Estrategia general

La plataforma debe implementar:

- HTTPS.
 - JWT/Auth seguro.
 - Roles.
 - Políticas.
 - Validaciones.
 - Protección de archivos.
-

Seguridad backend

Row Level Security

Utilizar RLS en PostgreSQL/Supabase.

Políticas

Definir políticas por:

- usuario.
 - recurso.
 - ownership.
 - roles.
-

Seguridad uploads

Validar:

- tamaño.
 - formato.
 - contenido.
-

12. OBSERVABILIDAD

Logging

Registrar:

- errores.
 - eventos críticos.
 - acciones admin.
-

Analytics

Medir:

- retención.
 - engagement.
 - IA.
 - feed.
 - chats.
-

Crash monitoring

Utilizar:

- Crashlytics.
 - Error reporting.
-

13. ESCALABILIDAD FUTURA

Estrategia inicial

Monolito modular.

Evolución futura posible

Extraer posteriormente:

- AI service.
 - recommendation engine.
 - video processing.
 - analytics.
-

Importante

NO iniciar con:

- microservicios.
 - Kubernetes.
 - arquitectura distribuida compleja.
-

14. ESTRATEGIA DEVOPS

Entornos

```
dev
staging
production
```

CI/CD

Automatizar:

- testing.
 - linting.
 - build.
 - deploy.
-

Secrets

Gestionar mediante:

- environment variables.
 - secret manager.
-

15. PRINCIPIOS DE DESARROLLO

Buenas prácticas obligatorias

Arquitectura

- Clean Architecture.
 - SOLID.
 - DRY.
 - Separation of Concerns.
-

Código

- Tipado fuerte.
 - Código modular.
 - Componentes reutilizables.
 - Feature-first.
-

UI/UX

- Mobile-first.
 - UI premium.
 - Animaciones fluidas.
 - Diseño minimalista.
-

16. ROADMAP ARQUITECTÓNICO

Fase 1 — MVP

- Feed.

- Auth.
 - Chat.
 - IA básica.
 - Verificación.
-

Fase 2 — Escalabilidad

- Optimización feed.
 - Mejoras realtime.
 - Recommendation engine.
-

Fase 3 — IA avanzada

- IA 3D.
 - Render pipeline.
 - Workers.
-

Fase 4 — Ecosistema avanzado

- Marketplace.
 - Payments.
 - AR/VR.
 - Streaming.
-

17. CONCLUSIÓN

La arquitectura de joyerIA está diseñada para:

- Construir rápidamente.
- Escalar progresivamente.
- Mantener alta calidad técnica.
- Minimizar complejidad inicial.
- Ofrecer experiencia premium.

El enfoque principal será:

- Monolito modular.
- Flutter + Supabase.
- Arquitectura limpia.

- Feature-first.
- IA desacoplada.
- PostgreSQL como núcleo.

La prioridad NO será construir infraestructura enterprise prematura, sino crear una base sólida, mantenible y preparada para evolucionar junto con el crecimiento real del producto.